a) Create mentioned Tables in SQL with proper primary and foreign keys. (2x5)

```sql
CREATE DATABASE Assignment1

USE Assignment1

CREATE TABLE Users (
   user_id INT PRIMARY KEY,
   username VARCHAR(50) NOT NULL UNIQUE,
   email VARCHAR(100) NOT NULL UNIQUE,
   dob DATE NOT NULL,
   created_at DATE
);

CREATE TABLE Books (
   book_id INT PRIMARY KEY,
   title VARCHAR(200) NOT NULL,
   author VARCHAR(100) NOT NULL,
   published_date DATE NOT NULL,
   genre VARCHAR(50) NOT NULL,
   created_at DATE
);

CREATE TABLE Chapters (
   chapter_id INT PRIMARY KEY,
   book_id INT,
   chapter_number INT NOT NULL,
   title VARCHAR(200) NOT NULL,
   content TEXT NOT NULL,
   total_words INT NOT NULL,
   FOREIGN KEY (book_id) REFERENCES Books(book_id)
);

CREATE TABLE Reviews (
   review_id INT PRIMARY KEY,
   user_id INT,
   book_id INT,
   rating INT CHECK (rating >= 1 AND rating <= 5),
   review_text TEXT NOT NULL,
   created_at DATE,
   FOREIGN KEY (user_id) REFERENCES Users(user_id),
   FOREIGN KEY (book_id) REFERENCES Books(book_id)
);

CREATE TABLE Liked_Books (
   like_id INT PRIMARY KEY,
   user_id INT,
   book_id INT,
```

```sql
    liked_at DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
```

b) **Insert at least 5 rows in each table. (1x5)**

```sql
INSERT INTO Users (user_id, username, email, dob, created_at) VALUES
(1, 'john_doe', 'john@email.com', '1990-05-15', '2023-01-15'),
(2, 'jane_smith', 'jane@email.com', '1995-08-22', '2023-03-22'),
(3, 'ali_khan', 'ali@email.com', '2005-03-10', '2023-06-10'),
(4, 'sara_ahmed', 'sara@email.com', '2000-11-30', '2023-08-05'),
(5, 'mike_wilson', 'mike@email.com', '1988-12-25', '2023-09-05'),
(6, 'teen_reader', 'teen@email.com', '2008-01-01', '2023-10-20'),
(7, 'bob_brown', 'bob@email.com', '1975-06-15', '2023-11-05'),
(8, 'emma_davis', 'emma@email.com', '2010-12-01', '2023-12-18'),
(9, 'david_miller', 'david@email.com', '1985-03-20', '2024-01-07'),
(10, 'sophia_lee', 'sophia@email.com', '2007-09-10', '2024-01-25');


INSERT INTO Books (book_id, title, author, published_date, genre, created_at) VALUES
(1, 'The Adventure Begins', 'Ali', '2010-01-01', 'Adventure', '2023-02-10'),
(2, 'Mystery Manor', 'Sarah Williams', '2015-06-15', 'Mystery', '2023-04-15'),
(3, 'Comic Adventures', 'Mike Brown', '1960-03-20', 'Comics', '2023-06-20'),
(4, 'The Great Journey', 'Ali', '2012-09-10', 'Adventure', '2023-07-30'),
(5, 'Technical Guide', 'John Smith', '2020-12-05', 'Education', '2023-09-05'),
(6, 'Epic Tales', 'Ali', '2019-05-20', 'Fantasy', '2023-10-12'),
(7, 'Comic World', 'Mike Brown', '1965-07-15', 'Comics', '2023-11-25'),
(8, 'Long Story', 'Jane Doe', '2018-11-30', 'Novel', '2023-12-30'),
(9, 'Word Master', 'Ali', '2021-02-28', 'Education', '2024-01-15'),
(10, 'Ancient Tales', 'Sarah Williams', '1950-04-10', 'Fantasy', '2024-02-01');


INSERT INTO Reviews (review_id, user_id, book_id, rating, review_text, created_at)
VALUES
(1, 1, 1, 4, 'Great book!', '2023-03-15'),
(2, 1, 2, 5, 'Absolutely loved it', '2023-04-20'),
(3, 2, 2, 3, 'Decent read', '2023-05-25'),
(4, 3, 3, 4, 'Very entertaining', '2023-06-30'),
(5, 4, 4, 5, 'Highly recommended', '2023-07-10'),
(6, 5, 2, 4, 'Interesting plot', '2023-08-15'),
```

```
(7, 6, 2, 5, 'Amazing story', '2023-09-20'),
(8, 7, 2, 4, 'Well written', '2023-10-25'),
(9, 1, 3, 3, 'Good comic', '2023-11-30'),
(10, 2, 3, 4, 'Fun read', '2023-12-05'),
(11, 6, 3, 5, 'Best comic ever', '2024-01-10'),
(12, 8, 1, 4, 'Nice adventure', '2024-02-05');


INSERT INTO Liked_Books (like_id, user_id, book_id, liked_at) VALUES
(1, 1, 3, '2023-03-20'),
(2, 1, 5, '2023-04-25'),
(3, 3, 2, '2023-05-30'),
(4, 3, 3, '2023-07-05'),
(5, 7, 4, '2023-08-10'),
(6, 1, 9, '2023-09-15'),
(7, 2, 3, '2023-10-20'),
(8, 5, 9, '2023-11-25'),
(9, 5, 2, '2023-12-30'),
(10, 4, 3, '2024-01-05'),
(11, 5, 4, '2024-01-15'),
(12, 10, 5, '2024-01-25'),
(13, 6, 3, '2024-02-01'),
(14, 7, 10, '2024-02-10');

INSERT INTO Chapters (chapter_id, book_id, chapter_number, title, content,
total_words) VALUES
(1, 1, 1, 'Beginning', 'Content...', 2500),
(2, 1, 2, 'Middle', 'Content...', 3000),
(3, 1, 3, 'Climax', 'Content...', 2800),
(4, 1, 4, 'Resolution', 'Content...', 2600),
(5, 1, 5, 'Ending', 'Content...', 2700),
(6, 1, 6, 'Epilogue', 'Content...', 2400),
(7, 2, 1, 'Mystery Begins', 'Content...', 250),
(8, 2, 2, 'The Clue', 'Content...', 250),
(9, 2, 3, 'The Search', 'Content...', 250),
(10, 2, 4, 'The Discovery', 'Content...', 250),
(11, 2, 5, 'The Revelation', 'Content...', 250),
(12, 2, 6, 'The Chase', 'Content...', 250),
(13, 2, 7, 'The Trap', 'Content...', 250),
(14, 2, 8, 'The Escape', 'Content...', 250),
(15, 2, 9, 'The Truth', 'Content...', 250),
(16, 2, 10, 'The Confrontation', 'Content...', 250),
(17, 2, 11, 'The Resolution', 'Content...', 250),
(18, 2, 12, 'The End', 'Content...', 250),
(19, 3, 1, 'Comic Start', 'Content...', 1500),
(20, 3, 2, 'Comic Middle', 'Content...', 1600),
(21, 3, 3, 'Comic End', 'Content...', 1700);
```

QUESTION#02

a) List the books liked and reviewed by the same user.

```
SELECT DISTINCT b.title AS Book_Names
FROM (Liked_Books AS l JOIN Reviews AS r ON l.user_id = r.user_id AND l.book_id =
r.book_id) JOIN Books b ON l.book_id = b.book_id;
```

Results Messages

	Book_Names
1	Comic Adventures
2	Mystery Manor

b) Identify the users who liked books published at least 20 years before their birth Year.

```
SELECT DISTINCT username
FROM (Users AS U JOIN Liked_Books AS L ON U.user_id = L.user_id) JOIN Books AS B ON
B.book_id = L.book_id
WHERE YEAR(U.dob)-20 >= YEAR(published_date)
```

Results Messages

	username
1	ali_khan
2	bob_brown
3	jane_smith
4	john_doe
5	sara_ahmed
6	teen_reader

c) List all books neither liked nor reviewed by the same user.

```
SELECT B.title AS Book_Names
FROM (Books AS B LEFT JOIN Reviews AS R ON B.book_id = R.book_id)
where user_id IS NULL
INTERSECT
```

```
SELECT B.title AS Book_Names
FROM (Books AS B LEFT JOIN Liked_Books AS L ON B.book_id = L.book_id)
where user_id IS NULL
```

Results Messages

	Book_Names
1	Comic World
2	Epic Tales
3	Long Story

d) Identify the users who liked and reviewed At least one book in common.

```
SELECT  Distinct L.book_id, U.username
FROM (Liked_Books AS L JOIN Reviews AS r ON L.user_id = r.user_id AND L.book_id =
r.book_id) Join Users as U on U.user_id = R.user_id
```

Results Messages

	book_id	username
1	2	mike_wilson
2	3	ali_khan
3	3	jane_smith
4	3	john_doe
5	3	teen_reader

e) List users who have not liked and reviewed any book.

```
SELECT username AS Names
FROM (Users AS U LEFT JOIN Liked_Books AS L ON U.user_id = L.user_id) LEFT JOIN
Reviews AS R ON U.user_id = R.user_id
where L.like_id IS NULL AND R.review_id is NULL
```

177 Question#02(f)

Results Messages

	Names
1	david_miller

f) List users who have liked a book but have not reviewed it on the same date.

```
SELECT username AS Names, l.like_id, r.review_id
FROM (Liked_Books AS l JOIN Reviews AS r ON l.user_id = r.user_id AND l.book_id =
r.book_id) Join Users AS U ON U.user_id = l.user_id
WHERE l.liked_at <> r.created_at
```

Results Messages

	Names	like_id	review_id
1	john_doe	1	9
2	ali_khan	4	4
3	jane_smith	7	10
4	mike_wilson	9	6
5	teen_reader	13	11

g) Find books that have been either reviewed or liked, but not both.

```
SELECT Distinct B.title
FROM (Liked_Books AS L FULL JOIN Reviews AS R ON L.book_id = R.book_id) JOIN Books AS
B ON B.book_id = R.book_id
WHERE (like_id IS NOT NULL AND review_id IS NULL) OR (like_id IS NULL AND review_id IS
NOT NULL);
```

Results Messages

	title
1	The Adventure Begins

Results grid

h) Find books that have been both reviewed and have at least one chapter.

```
SELECT Distinct B.title
FROM (Reviews AS R JOIN Chapters AS C ON R.book_id = C.book_id) JOIN Books as B ON
B.book_id = R.book_id;
```

Results Messages

	title
1	Comic Adventures
2	Mystery Manor
3	The Adventure Begins

Results

QUESTION#03

a) Retrieve all books that have more than 5 chapters.

```
SELECT B.title
FROM Chapters AS C Join Books AS B ON C.book_id = B.book_id
Group By B.title
HAVING count(C.book_id)>5
```

Results Messages

	title
1	Mystery Manor
2	The Adventure Begins

b) List the top 3 most-reviewed books with their total number of reviews.

```
SELECT TOP 3 count(B.book_id) AS Total_Reviews, B.title
FROM Reviews AS R JOIN Books AS B ON R.book_id = B.book_id
Group By B.title
ORDER BY Total_Reviews DESC
```

Results Messages

	Total_Reviews	title
1	5	Mystery Manor
2	4	Comic Adventures
3	2	The Adventure Begins

c) Find the average rating of books authored by "Ali."

```
SELECT AVG(R.rating) AS Average
FROM Reviews AS R  JOIN Books AS B ON R.book_id = B.book_id
WHERE B.author = 'Ali'
```

Results Messages

	Average
1	4

d) Identify the users who liked at least 5 books.

```
SELECT U.username, COUNT(L.like_id) AS Total_Likes
FROM Users AS U
JOIN Liked_Books AS L ON U.user_id = L.user_id
GROUP BY U.username
HAVING COUNT(L.like_id) > 5;
```

Results Messages

	username	Total_Likes

R

e) List the review of books with at least 10+ chapters.

```
SELECT B.title, CAST(R.review_text AS VARCHAR(MAX)) AS review_text
FROM Books AS B
JOIN Reviews AS R ON B.book_id = R.book_id
JOIN Chapters AS C ON B.book_id = C.book_id
```

```
GROUP BY B.book_id, B.title, CAST(R.review_text AS VARCHAR(MAX))
HAVING MAX(C.chapter_number) >= 10;
```

Results Messages

	title	review_text
1	Mystery Manor	Absolutely loved it
2	Mystery Manor	Amazing story
3	Mystery Manor	Decent read
4	Mystery Manor	Interesting plot
5	Mystery Manor	Well written

f) Find the average rating of teens (age 13 to 19, both inclusive) and the average rating of non-teens.

```
SELECT
   AVG(CASE WHEN 2025 - YEAR(U.dob) BETWEEN 13 AND 19 THEN R.rating END) AS
Teen_Avg_Rating,
   AVG(CASE WHEN 2025 - YEAR(U.dob) NOT BETWEEN 13 AND 19 THEN R.rating END) AS
Non_Teen_Avg_Rating
FROM Users AS U
JOIN Reviews AS R ON U.user_id = R.user_id;
```

	Teen_Avg_Rating	Non_Teen_Avg_Rating
1	4	4

g) List the average age of users who like comic books.

```
SELECT AVG(YEAR(CURDATE()) - YEAR(U.dob))  AS Average_age
FROM (Users AS U JOIN Liked_Books AS L ON U.user_id = L.user_id) JOIN Books AS B ON
L.book_id = B.book_id
WHERE B.genre = 'Comics';
```

Results Messages

	Average_age
1	25

Results grid

h) List usernames of users who have liked and reviewed the book whose average word length in each chapter is 250.

```
--Question#03(H)--
SELECT U.username
FROM (Users AS U JOIN Liked_Books AS L ON U.user_id = L.user_id) JOIN Reviews AS R ON
U.user_id = R.user_id AND L.book_id = R.book_id JOIN Chapters AS C ON R.book_id =
C.book_id
GROUP BY U.username, C.book_id
HAVING AVG(C.total_words) = 250;
```

Results Messages

	username
1	mike_wilson

i) Find the percentage of books that have been reviewed.

```
SELECT (COUNT(DISTINCT R.book_id))*100/ COUNT(DISTINCT B.book_id)AS
Reviewed_Percentage
FROM Books AS B LEFT JOIN Reviews AS R ON B.book_id = R.book_id;
```

	Reviewed_Percentage
1	40